

Project: Il modello di March

apprendimento organizzativo e dinamiche stocastiche

1. Obiettivi della dispensa

Questa dispensa introduce il modello di March dell'apprendimento organizzativo come caso di studio per un corso di metodi computazionali per modelli stocastici.

Gli obiettivi sono quattro:

1. comprendere la struttura del modello;
2. formalizzarne la dinamica in termini probabilistici;
3. identificare osservabili quantitative misurabili in simulazione;
4. usare il modello come base per attività di laboratorio e progetti computazionali.

Il modello è importante perché mostra come un sistema collettivo possa apprendere, o fallire nell'apprendere, a partire dall'interazione tra individui eterogenei e un codice organizzativo condiviso.

2. Idea generale del modello

Il punto di partenza è semplice: un'organizzazione cerca di apprendere una realtà esterna composta da molte proposizioni binarie. Gli individui possiedono credenze parziali, talvolta corrette, talvolta sbagliate, talvolta assenti. L'organizzazione, però, non coincide con la semplice somma delle persone che la compongono: esiste anche un codice condiviso, o codice organizzativo, che rappresenta la conoscenza pubblica e istituzionalizzata.

La dinamica ha due direzioni:

- gli individui apprendono dal codice;
- il codice apprende dalla popolazione.

L'interazione tra queste due scale genera il processo di apprendimento organizzativo.

3. Definizione formale

3.1 La realta' esterna

La realta' esterna e' descritta da un vettore di m proposizioni:

$$r = (r_1, \dots, r_m), \quad r_k \in \{-1, 1\}.$$

Ogni componente r_k rappresenta lo stato vero della proposizione k .

3.2 Gli individui

L'organizzazione contiene n individui. Ogni individuo i possiede un vettore di credenze:

$$b_i = (b_{i1}, \dots, b_{im}), \quad b_{ik} \in \{-1, 0, 1\}.$$

Interpretazione:

- $b_{ik} = 1$ significa che l'individuo ritiene vera la proposizione k ;
- $b_{ik} = -1$ significa che la ritiene falsa;
- $b_{ik} = 0$ significa assenza di credenza o ignoranza.

3.3 Il codice organizzativo

Il codice organizzativo e' un vettore

$$c = (c_1, \dots, c_m), \quad c_k \in \{-1, 0, 1\}.$$

Anche qui:

- $c_k = 1$ indica che il codice afferma la proposizione;
- $c_k = -1$ indica che la nega;
- $c_k = 0$ indica che il codice non ha ancora incorporato una posizione definita.

4. Dinamica del modello

4.1 Aggiornamento degli individui

Gli individui apprendono dal codice con probabilita' p_1 .

Per ciascun individuo i e ciascuna proposizione k :

- se $b_{ik}(t) = c_k(t)$, non cambia nulla;
- se $b_{ik}(t) \neq c_k(t)$, allora l'individuo puo' adottare il valore del codice con probabilita' p_1 .

Una regola semplice e' quindi:

$$b_{ik}(t+1) = \begin{cases} c_k(t) & \text{con probabilita' } p_1 \text{ se } b_{ik}(t) \neq c_k(t), \\ b_{ik}(t) & \text{altrimenti.} \end{cases}$$

Questa regola formalizza la pressione del codice sugli individui.

4.2 Aggiornamento del codice

Il codice apprende dalla popolazione. Per ogni proposizione k , si costruisce un segnale aggregato:

$$s_k(t) = \sum_{i=1}^n b_{ik}(t).$$

Si definisce quindi il segnale prevalente:

$$v_k(t) = \begin{cases} 1 & \text{se } s_k(t) > 0, \\ 0 & \text{se } s_k(t) = 0, \\ -1 & \text{se } s_k(t) < 0. \end{cases}$$

Se il codice non coincide con il segnale prevalente, esso si aggiorna con probabilita' p_2 :

$$c_k(t+1) = \begin{cases} v_k(t) & \text{con probabilita' } p_2 \text{ se } c_k(t) \neq v_k(t), \\ c_k(t) & \text{altrimenti.} \end{cases}$$

Il parametro p_2 misura quindi la rapidita' con cui il codice recepisce l'informazione dispersa nella popolazione.

4.3 Natura stocastica del sistema

Lo stato completo del sistema al tempo t e' dato da

$$X_t = (B(t), c(t)),$$

dove $B(t)$ e' la matrice delle credenze individuali.

Poiche' gli aggiornamenti dipendono da probabilita', la successione $\{X_t\}_{t \geq 0}$ definisce una catena di Markov a tempo discreto.

Questo e' il punto centrale dal punto di vista del corso: il modello nasce da regole microscopiche probabilistiche e produce esiti collettivi osservabili a livello macroscopico.

5. Interpretazione dei parametri

I parametri fondamentali sono due:

- p_1 : velocita' con cui gli individui apprendono dal codice;
- p_2 : velocita' con cui il codice apprende dalla popolazione.

La loro relazione e' cruciale.

Se p_1 e' molto grande e p_2 piccolo, gli individui si allineano rapidamente a un codice che potrebbe essere ancora poco informativo. Questo puo' produrre conformismo precoce e bloccare l'apprendimento.

Se invece p_2 e' abbastanza grande, il codice puo' incorporare piu' rapidamente l'informazione utile presente nella popolazione e l'organizzazione puo' convergere verso stati piu' accurati.

Il modello rende quindi concreto il trade-off tra:

- esplorazione: mantenimento di diversita' cognitiva;
- sfruttamento: allineamento rapido a una conoscenza codificata.

6. Osservabili da misurare

Per analizzare la dinamica servono osservabili quantitative.

6.1 Accuratezza del codice

Misura la frazione di componenti del codice che coincidono con la realta':

$$A_c(t) = \frac{1}{m} \sum_{k=1}^m \mathbf{1}\{c_k(t) = r_k\}.$$

6.2 Accuratezza media degli individui

Misura quanto, in media, gli individui siano vicini alla realta':

$$A_b(t) = \frac{1}{n} \sum_{i=1}^n \frac{1}{m} \sum_{k=1}^m \mathbf{1}\{b_{ik}(t) = r_k\}.$$

6.3 Tasso di ignoranza

Misura la frazione di componenti nulle nelle credenze individuali:

$$I(t) = \frac{1}{nm} \sum_{i=1}^n \sum_{k=1}^m \mathbf{1}_{\{b_{ik}(t) = 0\}}.$$

6.4 Diversita' interna

Una misura semplice della diversita' e' la distanza media di Hamming tra individui:

$$D(t) = \frac{2}{n(n-1)} \sum_{1 \leq i < j \leq n} d_H(b_i(t), b_j(t)).$$

Questa osservabile e' importante per capire se l'organizzazione stia mantenendo una certa pluralita' di punti di vista oppure stia convergendo troppo rapidamente verso l'omogeneita'.

7. Domande scientifiche che il modello permette di studiare

Il modello e' utile non solo come esempio teorico, ma come strumento per rispondere a domande precise.

1. **Effetto dei parametri:** Come varia l'accuratezza finale del codice al variare di p_1 e p_2 ?
2. **Convergenza:** Quanto tempo impiega l'organizzazione a raggiungere uno stato quasi stazionario?
3. **Lock-in:** Il sistema puo' convergere a un codice stabile ma sbagliato rispetto alla realta'?
4. **Ruolo della diversita' iniziale:** Una popolazione inizialmente piu' eterogenea migliora o peggiora l'apprendimento collettivo?
5. **Relazione tra scala micro e scala macro:** Come si passa dalle regole probabilistiche dei singoli agenti alle proprieta' aggregate dell'organizzazione?

8. Pseudocodice del modello

Di seguito una versione semplice del modello con aggiornamento sincrono.

8.1 Input

- numero di individui n

- numero di proposizioni m
- probabilita' di apprendimento individuale p_1
- probabilita' di aggiornamento del codice p_2
- numero massimo di passi temporali T

8.2 Pseudocodice

Genera $r_k \in \{-1, 1\}$ per $k = 1, \dots, m$

Inizializza $b_{ik}(0) \in \{-1, 0, 1\}$

Inizializza $c_k(0) \in \{-1, 0, 1\}$

Per $t = 0, \dots, T - 1$:

1. Copia lo stato corrente in una nuova configurazione temporanea.
2. Per ogni individuo i :
 - per ogni proposizione k :
 - se $b_{ik}(t) \neq c_k(t)$:
 - * estrai un numero casuale $u \sim U(0, 1)$;
 - * se $u < p_1$, poni

$$b_{ik}(t+1) = c_k(t).$$

3. Per ogni proposizione k :
 - calcola

$$s_k(t) = \sum_{i=1}^n b_{ik}(t+1);$$

- determina $v_k(t)$ dal segno di $s_k(t)$;
- se $c_k(t) \neq v_k(t)$:
 - estrai un numero casuale $u \sim U(0, 1)$;
 - se $u < p_2$, poni

$$c_k(t+1) = v_k(t).$$

4. Calcola le osservabili:

$$A_c(t+1), \quad A_b(t+1), \quad I(t+1), \quad D(t+1).$$

5. Salva i risultati.

Ripeti per molte realizzazioni indipendenti e calcola le medie di ensemble.

9. Commento didattico sul pseudocodice

Dal punto di vista computazionale, il modello permette di esercitare competenze fondamentali:

- rappresentazione di vettori e matrici di stato;
- uso di generatori di numeri casuali;
- aggiornamento stocastico;
- calcolo di osservabili aggregate;
- confronto tra traiettorie singole e medie statistiche.

Inoltre il modello e' abbastanza semplice da essere implementato sia in Python sia in altri linguaggi senza difficolta' strutturali.

10. Schema del laboratorio

10.1 Laboratorio 1 - Implementazione del modello base

Obiettivo

Implementare il modello di March con aggiornamento sincrono e misurare le osservabili principali.

Attivita'

1. Fissare valori iniziali, ad esempio:

$$n = 50, \quad m = 30, \quad T = 100.$$

2. Generare casualmente la realta' esterna.
3. Inizializzare le credenze individuali con una distribuzione assegnata.
4. Inizializzare il codice.
5. Simulare una traiettoria temporale.
6. Rappresentare graficamente:
 - $A_c(t)$
 - $A_b(t)$
 - $I(t)$
 - eventualmente $D(t)$

Domande guida

- Il codice migliora nel tempo?
- Gli individui apprendono piu' velocemente del codice oppure il contrario?
- L'ignoranza iniziale si riduce sempre?
- Il sistema converge a uno stato stabile?

Output richiesto

- codice sorgente;
- grafici temporali;
- breve commento interpretativo.

10.2 Laboratorio 2 - Analisi parametrica

Obiettivo

Studiare come la dinamica dipenda da p_1 e p_2 .

Attività

1. Scegliere una griglia di valori, ad esempio:

$$p_1 \in \{0.1, 0.3, 0.5, 0.7, 0.9\}, \quad p_2 \in \{0.1, 0.3, 0.5, 0.7, 0.9\}.$$

2. Per ogni coppia (p_1, p_2) , eseguire molte simulazioni indipendenti.
3. Calcolare il valore medio finale di $A_c(T)$.
4. Costruire una mappa dei risultati nel piano (p_1, p_2) .

Domande guida

- Esistono regioni dei parametri in cui il codice apprende bene?
- Esistono regioni in cui l'organizzazione si blocca?
- Un apprendimento individuale troppo rapido e' sempre vantaggioso?

Output richiesto

- tabella o heatmap dei risultati;
- commento sulle regioni del piano dei parametri;
- confronto tra casi favorevoli e casi sfavorevoli.

10.3 Laboratorio 3 - Condizioni iniziali e lock-in

Obiettivo

Studiare la dipendenza dalle condizioni iniziali.

Attività

1. Ripetere le simulazioni variando:
 - la quota iniziale di zeri;
 - la quota iniziale di credenze corrette;
 - la quota iniziale di credenze errate.
2. Confrontare i tempi di convergenza.

3. Verificare se configurazioni iniziali diverse portano a esiti finali diversi.

Domande guida

- Una maggiore diversita' iniziale aiuta?
- Il sistema puo' convergere a un codice sbagliato?
- Esiste memoria delle condizioni iniziali?

Output richiesto

- confronto tra scenari iniziali;
- grafici sovrapposti delle traiettorie;
- discussione del fenomeno di lock-in.

10.4 Laboratorio 4 - Estensioni

Obiettivo

Modificare il modello base per esplorare ipotesi piu' realistiche.

Possibili estensioni

1. eterogeneita' degli individui:

$$p_1 \rightarrow p_{1,i};$$

2. rumore nella trasmissione del codice;
3. turnover degli agenti;
4. aggiornamento asincrono;
5. influenza locale tramite rete sociale.

Domande guida

- L'eterogeneita' migliora la robustezza dell'organizzazione?
- Il rumore puo' paradossalmente evitare conformismo precoce?
- La rete sociale rafforza o indebolisce il ruolo del codice?

11. Esercizi consigliati

- **Esercizio 1** Implementare il modello base e verificare numericamente che $A_c(t)$ e $A_b(t)$ non coincidono in generale.
- **Esercizio 2** Stimare il tempo medio di convergenza al variare di p_1 mantenendo fisso p_2 .
- **Esercizio 3** Costruire una heatmap di $A_c(T)$ nel piano (p_1, p_2) .
- **Esercizio 4** Confrontare aggiornamento sincrono e asincrono.
- **Esercizio 5** Aggiungere eterogeneita' individuale e discutere come cambia la dinamica.

12. Una possibile estensione teorica: descrizione media

Per scopi didattici si puo' introdurre una descrizione media molto semplice.

Sia $x(t)$ la probabilita' media che una componente del codice sia corretta e $y(t)$ la probabilita' media che una credenza individuale sia corretta. Una chiusura qualitativa possibile e':

$$y(t+1) - y(t) \approx p_1(x(t) - y(t)),$$

$$x(t+1) - x(t) \approx p_2(F(y(t)) - x(t)),$$

dove $F(y)$ rappresenta l'effetto aggregato della popolazione sul codice.

Questa descrizione non e' esatta, ma e' utile per mostrare agli studenti la differenza tra:

- simulazione microscopica;
- osservabili macroscopiche;
- approssimazione analitica.

13. Perche' questo modello e' un buon caso di studio

Il modello di March e' particolarmente adatto per una dispensa didattica per tre motivi.

Primo, ha un forte contenuto interdisciplinare: puo' essere letto in termini di apprendimento organizzativo, diffusione di norme, formazione del consenso, trasmissione culturale e sistemi collettivi.

Secondo, e' un modello genuinamente stocastico ma facile da simulare.

Terzo, permette di introdurre molti strumenti centrali del corso:

- simulazione Monte Carlo;
- catene di Markov;
- osservabili collettive;
- dipendenza dalle condizioni iniziali;
- analisi parametrica;
- confronto tra dinamica microscopica e fenomeni emergenti.

14. Conclusione

Il modello di March mostra che l'apprendimento collettivo non dipende solo dalla qualità iniziale delle informazioni, ma anche dalla struttura del processo di aggiornamento. Un'organizzazione può diventare rapidamente coerente ma sbagliata, oppure mantenere abbastanza diversità da apprendere meglio nel lungo periodo.

Dal punto di vista didattico, questo modello è quindi un eccellente punto di partenza per un corso di metodi computazionali per modelli stocastici: è semplice da implementare, ricco dal punto di vista concettuale e adatto a molte estensioni.

15. Bibliografia minima

1. March, J. G. (1991). Exploration and Exploitation in Organizational Learning. *Organization Science*, 2(1), 71-87.
2. Levitt, B., and March, J. G. (1988). Organizational Learning. *Annual Review of Sociology*, 14, 319-340.
3. Arthur, W. B. (1994). Increasing Returns and Path Dependence in the Economy. University of Michigan Press.
4. Epstein, J. M. (2006). Generative Social Science. Princeton University Press.

Appendice A. Applicazioni empiriche e calibrazione del modello di March

Il modello di March nasce come modello generativo di apprendimento organizzativo. Il suo scopo originario non è quello di fornire una stima econometrica immediata, ma di rappresentare in modo semplice e trasparente il rapporto tra conoscenza distribuita negli individui e conoscenza incorporata nelle strutture organizzative. Proprio per questo, il modello è molto utile sia come strumento concettuale sia come base per simulazioni calibrate su dati osservabili.

A.1 Applicazioni concrete del modello

Il modello può essere usato in modo concreto in tutti i contesti in cui un'organizzazione deve apprendere, selezionare e consolidare informazione.

Esempi naturali sono:

- organizzazioni pubbliche che aggiornano procedure e linee guida;
- imprese che devono recepire conoscenza tecnica distribuita nei team;
- università e centri di ricerca che trasformano conoscenza individuale in pratiche istituzionali;

- ospedali e strutture sanitarie che aggiornano protocolli sulla base di esperienza e risultati;
- organizzazioni che introducono nuove tecnologie e devono coordinare apprendimento locale e policy centrali.

In questi contesti, il modello non va interpretato come fotografia letterale della realtà, ma come schema dinamico che aiuta a capire quando l'organizzazione apprende, quando converge troppo presto, e quando invece rimane bloccata su configurazioni poco accurate.

A.2 Che cosa significa “calibrare” il modello

Calibrare il modello significa associare le sue variabili teoriche a quantità osservabili o almeno misurabili tramite proxy.

Nel modello compaiono quattro oggetti fondamentali:

1. la realtà esterna r ;
2. le credenze individuali b_i ;
3. il codice organizzativo c ;
4. i parametri di aggiornamento p_1 e p_2 .

Per passare ai dati occorre quindi costruire una corrispondenza empirica.

Realta' esterna

La realtà esterna può essere interpretata come:

- una classificazione corretta;
- uno standard tecnico noto;
- un benchmark di performance;
- un insieme di decisioni ex post riconosciute come corrette;
- un target operativo verificabile.

In termini pratici, r_k rappresenta la risposta corretta alla proposizione k .

Credenze individuali

Le credenze individuali possono essere approssimate da:

- survey ripetute nel tempo;
- risposte a questionari;
- valutazioni di casi;
- decisioni prese da singoli operatori;
- log di classificazione o diagnosi;
- testi prodotti dai membri dell'organizzazione.

In questo modo si ottiene una proxy empirica di b_{ik} .

Codice organizzativo

Il codice organizzativo può essere rappresentato da:

- manuali interni;
- procedure formali;
- linee guida operative;
- policy aziendali;
- documentazione tecnica ufficiale;
- knowledge base istituzionali;
- comunicazioni organizzative ricorrenti.

In tutti questi casi il codice non coincide con l'opinione di un singolo individuo, ma con ciò che l'organizzazione dichiara, insegna o stabilizza come conoscenza valida.

A.3 Interpretazione empirica dei parametri

Una volta costruite le corrispondenze tra modello e dati, i parametri possono essere letti in modo operativo.

Il parametro p_1 può essere interpretato come probabilità che un individuo si riallinei al codice quando è in disaccordo con esso.

In termini empirici, se al tempo t osserviamo che

$$b_{ik}(t) \neq c_k(t),$$

allora p_1 misura la probabilità che al tempo successivo valga

$$b_{ik}(t+1) = c_k(t).$$

Il parametro p_2 può invece essere interpretato come probabilità che il codice incorpori il segnale prevalente proveniente dalla popolazione.

Se definiamo il segnale aggregato

$$v_k(t) = \begin{cases} 1 & \text{se } \sum_i b_{ik}(t) > 0, \\ 0 & \text{se } \sum_i b_{ik}(t) = 0, \\ -1 & \text{se } \sum_i b_{ik}(t) < 0, \end{cases}$$

allora p_2 misura la probabilità che

$$c_k(t+1) = v_k(t)$$

quando il codice non coincide già con il segnale aggregato.

A.4 Dati necessari

Per una calibrazione credibile servono dati longitudinali, cioè osservazioni ripetute nel tempo.

Idealmente occorrono:

- misure delle credenze individuali in più istanti temporali;
- una ricostruzione del codice organizzativo negli stessi istanti;
- una definizione operativa della realtà esterna o del benchmark corretto;
- una scansione temporale coerente con la dinamica di aggiornamento.

Senza una dimensione temporale, diventa molto difficile distinguere allineamento, apprendimento e semplice correlazione statica.

A.5 Strategie di calibrazione

A seconda della qualità dei dati, si possono seguire strategie diverse.

Calibrazione diretta

Se si osservano direttamente credenze individuali e codice nel tempo, si possono stimare le frequenze di aggiornamento.

Per esempio, una stima empirica di p_1 è:

$$\hat{p}_1 = \frac{\text{numero di casi in cui } b_{ik}(t+1) = c_k(t) \text{ dato che } b_{ik}(t) \neq c_k(t)}{\text{numero di casi in cui } b_{ik}(t) \neq c_k(t)}.$$

In modo analogo, una stima di p_2 è:

$$\hat{p}_2 = \frac{\text{numero di casi in cui } c_k(t+1) = v_k(t) \text{ dato che } c_k(t) \neq v_k(t)}{\text{numero di casi in cui } c_k(t) \neq v_k(t)}.$$

Questa è la forma più semplice di tuning del modello sui dati.

Calibrazione tramite momenti simulati

Se le variabili non sono osservate perfettamente ma si possono confrontare alcune statistiche aggregate, allora si possono simulare molte versioni del modello e scegliere i parametri che riproducono meglio i dati.

Per esempio si possono confrontare:

- accuratezza finale del codice;
- tempo medio di convergenza;
- livello di diversità residua;
- frequenza di lock-in;

- velocità di riduzione dell'ignoranza.

In questo caso si scelgono i parametri che minimizzano una distanza tra statistiche simulate e statistiche osservate.

Calibrazione con variabili latenti

Se credenze individuali o codice sono solo parzialmente osservabili, allora il problema diventa uno di inferenza con variabili latenti. In questi casi è possibile usare approcci più avanzati, come:

- modelli hidden Markov;
- modelli state-space;
- Approximate Bayesian Computation;
- indirect inference.

Per una dispensa introduttiva è sufficiente sapere che questi metodi esistono e diventano necessari quando il legame tra modello e dati è indiretto.

A.6 Validazione del modello

Una volta calibrato, il modello non va giudicato solo in base alla bontà dell'adattamento interno, ma anche in base alla sua capacità di riprodurre regolarità osservate.

Una validazione ragionevole può includere:

- confronto tra traiettorie simulate e traiettorie osservate;
- confronto tra tempi di convergenza;
- confronto tra distribuzioni finali di accuratezza;
- capacità di riprodurre fenomeni di persistenza o lock-in;
- robustezza dei risultati al variare delle condizioni iniziali.

In altre parole, il modello va trattato come un modello generativo: non basta che “stia sui dati”, deve anche riprodurre meccanismi plausibili.

A.7 Limiti della calibrazione

La calibrazione del modello di March presenta alcune difficoltà strutturali.

Primo, il codice organizzativo non è sempre direttamente osservabile.

Secondo, le credenze individuali spesso devono essere inferite da testi, survey o comportamenti, e quindi contengono rumore.

Terzo, la realtà esterna non è sempre definibile in modo netto: in molti problemi organizzativi non esiste una verità binaria perfettamente osservabile.

Quarto, il modello base è intenzionalmente semplice, quindi molte organizzazioni reali richiedono estensioni che includano reti sociali, turnover, gerarchie, rumore di trasmissione o ambienti mutevoli.

Per questo motivo, nella pratica è spesso più corretto parlare di calibrazione data-informed o di tuning su proxy empiriche, piuttosto che di stima strutturale completa del modello originale.

A.8 Perché questa appendice è importante

Dal punto di vista didattico, il passaggio dal modello teorico ai dati è fondamentale. Mostra che un modello stocastico non serve solo a simulare scenari astratti, ma può anche essere collegato a evidenza empirica, purché si chiariscano bene:

- quali variabili sono osservabili;
- quali sono latenti;
- quali parametri si vogliono stimare;
- quali statistiche si vogliono riprodurre.

Questo rende il modello di March particolarmente utile in un corso di metodi computazionali: gli studenti vedono sia la dimensione generativa del modello sia i problemi concreti della sua traduzione empirica.

A.9 Messaggio finale

Il modello di March non è, nella sua forma originaria, un modello pensato per una calibrazione automatica e immediata su dati grezzi. Tuttavia, può essere adattato in modo molto utile a contesti empirici reali se si costruiscono buone proxy per le credenze individuali, per il codice organizzativo e per la realtà esterna.

Per questo motivo, il suo valore non è solo teorico. È anche metodologico: insegna come si passa da una descrizione microscopica probabilistica a una simulazione, e da una simulazione a una possibile strategia di confronto con i dati.

Bibliografia per l'appendice A

5. Cyert, R. M., and March, J. G. (1963). A Behavioral Theory of the Firm. Prentice-Hall.
6. Argote, L., and Miron-Spektor, E. (2011). Organizational Learning: From Experience to Knowledge. *Organization Science*, 22(5), 1123-1137.
7. He, Z.-L., and Wong, P.-K. (2004). Exploration vs. Exploitation: An Empirical Test of the Ambidexterity Hypothesis. *Organization Science*, 15(4), 481-494.

8. Posen, H. E., and Levinthal, D. A. (2012). Chasing a Moving Target: Exploitation and Exploration in Dynamic Environments. *Management Science*, 58(3), 587-601.
 9. Gilbert, N., and Troitzsch, K. G. (2005). *Simulation for the Social Scientist*. Open University Press.
-

Appendice B. Il ruolo della realta' nel modello originale di March

Questa appendice chiarisce un punto essenziale del modello originale di March. Nella formulazione piu' fedele al modello, la realta' esterna non entra soltanto come benchmark usato per valutare ex post la qualita' del codice organizzativo e delle credenze individuali. Essa svolge invece una funzione dinamica cruciale, perche' determina quali individui siano, in ciascun istante, piu' informati del codice stesso e possano quindi contribuire al suo aggiornamento.

Questo punto e' importante anche dal punto di vista didattico. Se la realta' fosse usata soltanto come criterio di valutazione finale, il modello sarebbe molto vicino a una dinamica puramente endogena di costruzione del consenso. Nel modello originale, invece, l'organizzazione non e' chiusa su se stessa: il suo apprendimento resta ancorato al mondo esterno, anche se in modo indiretto e mediato.

B.1 Realtà esterna e accuratezza

La realta' esterna e' rappresentata da un vettore di proposizioni binarie

$$r = (r_1, \dots, r_m), \quad r_k \in \{-1, 1\}.$$

Ogni componente r_k rappresenta lo stato vero della proposizione k .

Gli individui possiedono credenze

$$b_i = (b_{i1}, \dots, b_{im}), \quad b_{ik} \in \{-1, 0, 1\},$$

mentre il codice organizzativo e' descritto da

$$c = (c_1, \dots, c_m), \quad c_k \in \{-1, 0, 1\}.$$

Per misurare la qualita' epistemica di un individuo rispetto alla realta' si puo' definire

$$K_i(t) = \sum_{k=1}^m \mathbf{1}\{b_{ik}(t) = r_k\},$$

cioe' il numero di proposizioni sulle quali l'individuo i e' corretto.

In modo analogo, la qualita' epistemica del codice e'

$$K_c(t) = \sum_{k=1}^m \mathbf{1}\{c_k(t) = r_k\}.$$

Queste due quantita' misurano la vicinanza alla realta' del singolo individuo e del codice organizzativo.

B.2 Il gruppo che influenza il codice

Il punto decisivo del modello originale e' che il codice non apprende indistintamente dall'intera popolazione. Esso apprende soltanto dagli individui che, in quel momento, possiedono una conoscenza della realta' superiore a quella del codice.

Si definisce quindi il gruppo superiore

$$S(t) = \{i : K_i(t) > K_c(t)\}.$$

Gli individui appartenenti a $S(t)$ sono coloro che, al tempo t , risultano piu' accurati del codice rispetto alla realta' esterna. Sono quindi questi individui, e non la popolazione nel suo complesso, a fornire il segnale rilevante per l'aggiornamento del codice.

In questo modo la realta' entra nella dinamica in modo selettivo: non corregge direttamente il codice, ma decide indirettamente chi abbia titolo epistemico per influenzarlo.

B.3 Segnale prevalente e aggiornamento del codice

Per ogni proposizione k , si puo' contare quanti individui del gruppo $S(t)$ sostengono il valore $+1$ e quanti sostengono il valore -1 :

$$M_k^+(t) = \sum_{i \in S(t)} \mathbf{1}\{b_{ik}(t) = 1\},$$

$$M_k^-(t) = \sum_{i \in S(t)} \mathbf{1}\{b_{ik}(t) = -1\}.$$

Da queste quantita' si ricava il segnale prevalente:

$$v_k(t) = \begin{cases} 1 & \text{se } M_k^+(t) > M_k^-(t), \\ -1 & \text{se } M_k^-(t) > M_k^+(t), \\ 0 & \text{se } M_k^+(t) = M_k^-(t). \end{cases}$$

Una regola semplice di aggiornamento del codice e' allora

$$c_k(t+1) = \begin{cases} v_k(t) & \text{con probabilita' } p_2 \text{ se } v_k(t) \neq 0 \text{ e } c_k(t) \neq v_k(t), \\ c_k(t) & \text{altrimenti.} \end{cases}$$

Il parametro p_2 controlla quindi la capacita' del codice di recepire il segnale proveniente dagli individui che risultano migliori di esso.

B.4 Come entra la realta' nella dinamica

Alla luce della costruzione precedente, si puo' dire che la realta' entra nel modello in tre modi distinti.

Primo, la realta' definisce l'accuratezza delle credenze individuali, perche' permette di stabilire quando un individuo sia corretto o errato.

Secondo, la realta' definisce l'accuratezza del codice organizzativo, permettendo di valutare quanto l'organizzazione sia vicina al mondo esterno.

Terzo, e soprattutto, la realta' seleziona il sottoinsieme di individui che puo' influenzare il codice, cioe' quelli che in un dato istante risultano piu' accurati del codice stesso.

Ne segue che il modello non descrive una dinamica puramente autoreferenziale. L'organizzazione non costruisce la propria conoscenza solo sulla base di processi interni di conformita' o di imitazione reciproca. Vi e' invece un ancoraggio alla realta', anche se tale ancoraggio non assume la forma di una correzione diretta e continua da parte del mondo esterno.

B.5 Ancoraggio indiretto e mediazione organizzativa

Uno degli aspetti piu' interessanti del modello e' proprio il carattere mediato dell'ancoraggio empirico.

Gli individui, nel corso della dinamica, non confrontano direttamente e in ogni istante le proprie credenze con la realta'. Allo stesso modo, il codice non osserva direttamente il mondo esterno per correggersi. Il collegamento con la realta' passa invece attraverso una mediazione organizzativa:

1. la realta' determina chi e' piu' accurato;
2. gli individui piu' accurati influenzano il codice;
3. il codice influenza l'intera popolazione.

Questo meccanismo distingue il modello di March sia dai modelli puramente autoreferenziali, sia dai modelli di apprendimento diretto da feedback esterno. La conoscenza organizzativa non nasce da una semplice somma di osservazioni individuali, ma da un processo di selezione, incorporazione e diffusione.

B.6 Conseguenze teoriche

Questa struttura ha conseguenze teoriche rilevanti.

Anzitutto, la presenza di individui inizialmente migliori del codice puo' essere decisiva per l'apprendimento organizzativo. Se tali individui vengono assorbiti troppo rapidamente dal codice, o se la loro informazione non riesce a essere incorporata in tempo, l'organizzazione puo' perdere una risorsa epistemica preziosa.

In secondo luogo, il modello mostra che il conformismo non e' sempre vantaggioso. Un apprendimento individuale troppo rapido, controllato da un valore elevato di p_1 , puo' ridurre la diversita' cognitiva prima che il codice abbia tratto beneficio dall'informazione distribuita nella popolazione.

In terzo luogo, il modello suggerisce che l'efficacia dell'apprendimento organizzativo dipende dal bilanciamento tra due processi:

- la velocita' con cui gli individui si allineano al codice;
- la velocita' con cui il codice recepisce il contributo degli individui piu' accurati.

Se questo bilanciamento e' sfavorevole, l'organizzazione puo' convergere verso un codice internamente coerente ma poco accurato rispetto alla realta'.

B.7 Differenza rispetto a una versione semplificata

Per scopi didattici, talvolta si introduce una versione semplificata del modello in cui il codice apprende dal segnale aggregato dell'intera popolazione. In quella versione la realta' entra soprattutto come benchmark esterno per misurare accuratezza, ma non determina direttamente chi influenzi il codice.

Questa semplificazione puo' essere utile in laboratorio, perche' rende il modello piu' facile da implementare e da spiegare. Tuttavia, essa modifica un punto concettuale importante: nel modello originale il codice non segue semplicemente la maggioranza, ma apprende da una sotto-popolazione selezionata in base alla sua maggiore accuratezza rispetto al mondo esterno.

E' quindi utile che gli studenti distinguano chiaramente tra:

- modello originale, in cui la realta' seleziona chi possa influenzare il codice;
- versione semplificata, in cui la realta' e' usata soprattutto come criterio di valutazione.

B.8 Rilevanza per il corso

Dal punto di vista di un corso di metodi computazionali per modelli stocastici, questo aspetto del modello di March e' particolarmente istruttivo. Esso mostra che una dinamica stocastica collettiva puo' essere:

- endogena nelle sue regole di aggiornamento;
- ma al tempo stesso ancorata a una realta' esterna tramite un criterio di selezione.

Questo aiuta gli studenti a comprendere una distinzione metodologica fondamentale tra:

- modelli di pura interazione sociale;
- modelli con feedback esterno diretto;
- modelli con feedback esterno indiretto e mediato.

Il modello di March appartiene precisamente a questa terza categoria.

B.9 Conclusione dell'appendice

La realta' esterna nel modello originale di March non e' un elemento decorativo, ne' un semplice parametro usato per valutare i risultati alla fine della simulazione. Essa svolge una funzione strutturale nella dinamica del sistema, perche' determina chi, in ogni istante, sia epistemicamente superiore al codice e possa contribuire al suo aggiornamento.

Per questo motivo, il modello non descrive un'organizzazione totalmente chiusa su se stessa. L'apprendimento organizzativo resta radicato nel mondo esterno, ma in modo indiretto, selettivo e mediato dal rapporto tra individui e codice. E' proprio questa combinazione tra ancoraggio alla realta' e mediazione organizzativa a rendere il modello di March cosi' interessante sia sul piano teorico sia sul piano computazionale.

Appendice C. Implementazione in Python: struttura del codice

Questa appendice propone una struttura semplice e leggibile per implementare il modello di March in Python. L'obiettivo non e' costruire un programma

“sofisticato”, ma fornire uno schema chiaro che possa essere letto anche come pseudocodice da chi usa altri linguaggi di programmazione.

La scelta è volutamente elementare:

- si usano liste e cicli espliciti;
- si evitano strutture troppo compatte o troppo “pythoniche”;
- si indicano esplicitamente le librerie minime da importare;
- il codice è abbastanza vicino a Python reale da poter essere implementato con sforzo minimo.

C.1 Librerie minime

Per una prima implementazione bastano le librerie standard di Python:

```
import random
import statistics
```

Se si vogliono anche grafici semplici delle traiettorie temporali, conviene aggiungere:

```
import matplotlib.pyplot as plt
```

Quindi:

- `random` serve per generare variabili casuali;
- `statistics` serve per medie e deviazioni standard semplici;
- `matplotlib.pyplot` serve per i grafici.

Non è necessario usare `numpy` in una prima versione. Se qualcuno lo conosce già, puo' introdurlo in un secondo momento per rendere il codice più veloce, ma non è indispensabile.

C.2 Come rappresentare i dati

Nel modello ci sono tre oggetti fondamentali:

1. la realtà esterna;
2. le credenze individuali;
3. il codice organizzativo.

Una rappresentazione molto semplice è la seguente.

Realtà esterna

La realtà è una lista di lunghezza m :

```
reality = [1, -1, 1, 1, -1]
```

In generale:

- `reality[k]` rappresenta il valore vero della proposizione `k`.

Credenze individuali

Le credenze della popolazione possono essere rappresentate come una lista di liste:

```
beliefs = [  
    [1, 0, -1, 1, 0],  
    [-1, 1, -1, 0, 1],  
    [0, 0, 1, -1, 1]  
]
```

Qui:

- `beliefs[i]` è il vettore delle credenze dell'individuo `i`;
- `beliefs[i][k]` è la credenza dell'individuo `i` sulla proposizione `k`.

Codice organizzativo

Anche il codice è una lista di lunghezza `m`:

```
code = [0, 0, 1, -1, 0]
```

C.3 Struttura generale del programma

Una buona organizzazione del codice è la seguente:

1. funzioni di inizializzazione;
2. funzioni di aggiornamento;
3. funzioni per calcolare le osservabili;
4. una funzione principale che esegue una simulazione;
5. una funzione esterna che ripete molte simulazioni indipendenti.

In altre parole, conviene separare bene:

- la logica del modello;
- la raccolta dei risultati;
- l'analisi statistica finale.

C.4 Inizializzazione

Generare la realtà

```
def create_reality(m):
    reality = []
    for k in range(m):
        value = random.choice([-1, 1])
        reality.append(value)
    return reality
```

Generare le credenze iniziali

Una scelta semplice è inizializzare ogni componente con probabilità uguale su -1, 0, 1:

```
def create_beliefs(n, m):
    beliefs = []
    for i in range(n):
        agent = []
        for k in range(m):
            value = random.choice([-1, 0, 1])
            agent.append(value)
        beliefs.append(agent)
    return beliefs
```

Se si vuole controllare meglio la distribuzione iniziale, si può costruire una funzione più flessibile. Per esempio:

```
def random_belief(p_minus, p_zero, p_plus):
    u = random.random()
    if u < p_minus:
        return -1
    elif u < p_minus + p_zero:
        return 0
    else:
        return 1
```

e poi usarla dentro `create_beliefs`.

Generare il codice iniziale

```
def create_code(m):
    code = []
    for k in range(m):
        value = random.choice([-1, 0, 1])
        code.append(value)
    return code
```

C.5 Misurare l'accuratezza

Per implementare il modello originale di March serve confrontare individui e codice con la realtà.

Accuratezza di un individuo

```
def individual_knowledge(agent, reality):
    score = 0
    m = len(reality)
    for k in range(m):
        if agent[k] == reality[k]:
            score += 1
    return score
```

Accuratezza del codice

```
def code_knowledge(code, reality):
    score = 0
    m = len(reality)
    for k in range(m):
        if code[k] == reality[k]:
            score += 1
    return score
```

C.6 Aggiornamento degli individui

Gli individui apprendono dal codice con probabilità $p1$.

```
def update_beliefs(beliefs, code, p1):
    n = len(beliefs)
    m = len(code)

    new_beliefs = []

    for i in range(n):
        new_agent = []
        for k in range(m):
            current_value = beliefs[i][k]
            code_value = code[k]

            if current_value != code_value:
                u = random.random()
                if u < p1:
                    new_agent.append(code_value)
            else:
```

```

        new_agent.append(current_value)
    else:
        new_agent.append(current_value)

    new_beliefs.append(new_agent)

    return new_beliefs

```

Osservazione importante: qui si costruisce una nuova lista `new_beliefs` invece di modificare direttamente `beliefs`. Questo rende l'aggiornamento sincrono e più facile da capire.

C.7 Costruire il gruppo superiore

Nel modello originale il codice apprende solo dagli individui più accurati del codice.

```

def superior_group(beliefs, code, reality):
    group = []

    code_score = code_knowledge(code, reality)

    for i in range(len(beliefs)):
        agent_score = individual_knowledge(beliefs[i], reality)
        if agent_score > code_score:
            group.append(i)

    return group

```

Qui `group` contiene gli indici degli individui che appartengono all'insieme $S(t)$.

C.8 Segnale prevalente del gruppo superiore

Per ogni proposizione `k` bisogna contare quanti membri del gruppo superiore sostengano `+1` e quanti `-1`.

```

def majority_signal_for_dimension(beliefs, group, k):
    plus_count = 0
    minus_count = 0

    for i in group:
        value = beliefs[i][k]
        if value == 1:
            plus_count += 1
        elif value == -1:
            minus_count += 1

```

```

if plus_count > minus_count:
    return 1
elif minus_count > plus_count:
    return -1
else:
    return 0

```

C.9 Aggiornamento del codice

Ora si può aggiornare il codice usando il gruppo superiore.

```

def update_code(beliefs, code, reality, p2):
    m = len(code)
    new_code = code.copy()

    group = superior_group(beliefs, code, reality)

    if len(group) == 0:
        return new_code

    for k in range(m):
        signal = majority_signal_for_dimension(beliefs, group, k)

        if signal != 0 and code[k] != signal:
            u = random.random()
            if u < p2:
                new_code[k] = signal

    return new_code

```

Questa funzione implementa l'idea essenziale del modello originale:

- si confrontano individui e codice con la realtà;
- si selezionano gli individui migliori del codice;
- il codice apprende solo da loro.

C.10 Osservabili principali

Accuratezza del codice

```

def code_accuracy(code, reality):
    correct = 0
    m = len(reality)

```

```

for k in range(m):
    if code[k] == reality[k]:
        correct += 1

return correct / m

```

Accuratezza media degli individui

```

def average_belief_accuracy(beliefs, reality):
    scores = []

    for agent in beliefs:
        score = individual_knowledge(agent, reality) / len(reality)
        scores.append(score)

    return statistics.mean(scores)

```

Tasso di ignoranza

```

def ignorance_rate(beliefs):
    total = 0
    zeros = 0

    for agent in beliefs:
        for value in agent:
            total += 1
            if value == 0:
                zeros += 1

    return zeros / total

```

Distanza di Hamming tra due individui

```

def hamming_distance(agent1, agent2):
    distance = 0
    m = len(agent1)

    for k in range(m):
        if agent1[k] != agent2[k]:
            distance += 1

    return distance

```

Diversità media della popolazione

```
def population_diversity(beliefs):
    n = len(beliefs)

    if n < 2:
        return 0.0

    distances = []

    for i in range(n):
        for j in range(i + 1, n):
            d = hamming_distance(beliefs[i], beliefs[j])
            distances.append(d)

    return statistics.mean(distances)
```

C.11 Una simulazione completa

Conviene ora costruire una funzione che esegua una singola traiettoria temporale.

```
def run_simulation(n, m, p1, p2, T):
    reality = create_reality(m)
    beliefs = create_beliefs(n, m)
    code = create_code(m)

    history_code_accuracy = []
    history_belief_accuracy = []
    history_ignorance = []
    history_diversity = []

    history_code_accuracy.append(code_accuracy(code, reality))
    history_belief_accuracy.append(average_belief_accuracy(beliefs, reality))
    history_ignorance.append(ignorance_rate(beliefs))
    history_diversity.append(population_diversity(beliefs))

    for t in range(T):
        beliefs = update_beliefs(beliefs, code, p1)
        code = update_code(beliefs, code, reality, p2)

        history_code_accuracy.append(code_accuracy(code, reality))
        history_belief_accuracy.append(average_belief_accuracy(beliefs, reality))
        history_ignorance.append(ignorance_rate(beliefs))
        history_diversity.append(population_diversity(beliefs))

    results = {
```

```

        "reality": reality,
        "beliefs": beliefs,
        "code": code,
        "code_accuracy": history_code_accuracy,
        "belief_accuracy": history_belief_accuracy,
        "ignorance": history_ignorance,
        "diversity": history_diversity
    }

    return results

```

Questa funzione restituisce un dizionario con:

- stato finale;
- traiettorie temporali delle osservabili.

C.12 Eseguire più simulazioni indipendenti

Per uno studio serio non basta una sola traiettoria. Occorre ripetere l'esperimento molte volte.

```

def run_many_simulations(num_runs, n, m, p1, p2, T):
    final_code_accuracies = []
    final_belief_accuracies = []
    final_ignorances = []
    final_diversities = []

    all_histories_code = []
    all_histories_belief = []

    for run in range(num_runs):
        results = run_simulation(n, m, p1, p2, T)

        final_code_accuracies.append(results["code_accuracy"][-1])
        final_belief_accuracies.append(results["belief_accuracy"][-1])
        final_ignorances.append(results["ignorance"][-1])
        final_diversities.append(results["diversity"][-1])

        all_histories_code.append(results["code_accuracy"])
        all_histories_belief.append(results["belief_accuracy"])

    summary = {
        "mean_final_code_accuracy": statistics.mean(final_code_accuracies),
        "mean_final_belief_accuracy": statistics.mean(final_belief_accuracies),
        "mean_final_ignorance": statistics.mean(final_ignorances),
        "mean_final_diversity": statistics.mean(final_diversities),
    }

```

```

        "all_histories_code": all_histories_code,
        "all_histories_belief": all_histories_belief
    }

    return summary

```

C.13 Come fare un grafico semplice

Se si usa `matplotlib`, si può disegnare facilmente una traiettoria:

```

def plot_single_run(results):
    times = list(range(len(results["code_accuracy"])))

    plt.plot(times, results["code_accuracy"], label="accuratezza codice")
    plt.plot(times, results["belief_accuracy"], label="accuratezza media individui")
    plt.plot(times, results["ignorance"], label="ignoranza")
    plt.xlabel("tempo")
    plt.ylabel("valore")
    plt.legend()
    plt.show()

```

Una possibile sequenza completa è:

```

n = 50
m = 30
p1 = 0.4
p2 = 0.6
T = 100

results = run_simulation(n, m, p1, p2, T)
plot_single_run(results)

```

C.14 Come organizzare un esperimento parametrico

Per studiare il ruolo di p_1 e p_2 si può usare un doppio ciclo.

```

p1_values = [0.1, 0.3, 0.5, 0.7, 0.9]
p2_values = [0.1, 0.3, 0.5, 0.7, 0.9]

table = []

for p1 in p1_values:
    row = []
    for p2 in p2_values:
        summary = run_many_simulations(

```

```

        num_runs=30,
        n=50,
        m=30,
        p1=p1,
        p2=p2,
        T=100
    )
    row.append(summary["mean_final_code_accuracy"])
    table.append(row)

```

Alla fine `table` contiene una matrice di risultati che può essere:

- stampata a schermo;
- trasformata in heatmap;
- esportata in un file.

C.15 Scelte di programmazione consigliate

Per una prima implementazione conviene seguire alcune regole semplici.

Usare nomi leggibili

Meglio scrivere:

```

beliefs
code
reality
ignorance_rate

```

piuttosto che abbreviazioni troppo corte e poco trasparenti.

Separare le funzioni

Ogni funzione dovrebbe fare una sola cosa:

- una funzione inizializza;
- una funzione aggiorna gli individui;
- una funzione aggiorna il codice;
- una funzione misura un'osservabile.

Questo rende il programma più facile da leggere, correggere e modificare.

Evitare ottimizzazioni premature

In una prima fase è meglio avere un codice più lento ma chiaro, piuttosto che un codice molto compatto ma difficile da capire.

Salvare i risultati in strutture semplici

Liste e dizionari sono sufficienti per quasi tutto il laboratorio introduttivo.

C.16 Versione semplificata per il primo laboratorio

Se si vuole partire con una versione ancora più semplice, si può temporaneamente modificare la funzione `update_code` facendo apprendere il codice dall'intera popolazione, invece che dal gruppo superiore.

In tal caso:

1. si calcola il segnale prevalente tra tutti gli individui;
2. il codice si aggiorna verso quel segnale con probabilità `p2`.

Questa versione è meno fedele al modello originale, ma può essere utile come primo esercizio. In un secondo momento si può sostituire con la versione completa basata sulla realtà e sul gruppo $S(t)$.

C.17 Possibili estensioni del codice

Una volta implementata la struttura base, il programma può essere esteso in molti modi.

Eterogeneità individuale

Invece di un unico parametro `p1`, si può usare una lista di probabilità individuali:

```
p1_list = [0.2, 0.8, 0.5, 0.3, ...]
```

Aggiornamento asincrono

Invece di aggiornare tutti gli individui in parallelo, si può scegliere casualmente un individuo alla volta.

Turnover

Si possono rimuovere alcuni individui e sostituirli con nuovi agenti.

Ambiente mutevole

La realtà `reality` può cambiare nel tempo.

Rete sociale

Si può aggiungere una struttura di vicinato tra individui, così che l'aggiornamento non dipenda solo dal codice.

C.18 Conclusione dell'appendice

La struttura proposta in questa appendice ha due obiettivi:

- essere abbastanza semplice da essere letta come pseudocodice anche da chi programma in altri linguaggi;
- essere abbastanza concreta da poter essere tradotta quasi direttamente in un programma Python funzionante.

Il messaggio metodologico è importante: per implementare un modello stocastico non serve partire da strumenti avanzati. Basta scomporre il problema in blocchi chiari:

1. rappresentazione dello stato;
2. regole di aggiornamento;
3. osservabili;
4. ripetizione Monte Carlo;
5. analisi dei risultati.

Una volta che questa struttura è chiara, il passaggio a versioni più ricche o più efficienti diventa molto più naturale.